Deterministic VS Randomised pivot selection in Quicksort

Hein Htet Aung - 25026511

Felix Herrmann - 25483227

Table of Contents

Table of Contents **1**
1. The problem and the algorithms **3**
1.1 The problem 3
1.2 The algorithms 3
1.2.1 Deterministic pivot selection 3
1.2.2 Randomised pivot selection 4
2. Theoretical comparison **5**
2.1 Asymptotic complexity 5
2.2 Constant factor 5
2.3 Memory characteristics 5
2.4 Input specific behaviour 5
3. Predictions **6**
3.1 Random 6
3.2 Sorted 6
3.3 Reverse sorted 6
3.4 Nearly sorted 6
3.5 Many duplicates 6
3.6 All equal 6
4. Experimental methodology **7**
4.1 Benchmark Inputs 7
4.2 Repetitions, Seeds, and Warm-up 8
4.3 Timing Procedure 8
4.4 Hardware and Compiler Configuration 9
4.5 Correctness Validation 9
4.6 Additional Algorithmic Measurements 10
4.6.1 Comparisons 10
4.6.2 Swaps 10
4.6.3 Maximum Recursion Depth 10
4.6.4 Initial Partition Imbalance 10
4.7 Output and Aggregation 11
4.8 Experimental Scope and Limitations 12
4.9 Flow Diagram of the Experiment algorithm 12
5. Results and analysis **13**
5.1 Deterministic pivots on sorted inputs 14
5.1.1 Similarity of deterministic and randomised algorithm within the same test 14
5.1.1.1 Significantly different: sorted, reverse sorted, nearly sorted 14
5.1.1.2 Mostly similar: all equal, many duplicates, random 16
5.1.2 Two tight runtime groupings 17
5.2 Algorithmic Measurements: Explaining the Observed Performance 17

6. What surprised us..... 19
6.1 Decreasing Execution Time Across Repetitions..... 19
6.2 Near-Quadratic Performance on Nearly Sorted Inputs..... 19
7. What we'd do next..... 20
7.1 Alternative deterministic pivot-selection strategies..... 20
7.2 Alternative partitioning schemes..... 20
7.3 Varying the degree of sortedness..... 20
7.4 Investigating duplicate density..... 21
7.5 Examining the relationship between initial partition and overall performance..... 21
7.6 Expanding performance measurements..... 21
8. Individual contributions..... 22
9. GitHub Repository Link..... 22
10. Video Link..... 22
11. AI use..... 23
11.1 Individual AI Use Disclosure..... 23
11.1.1 Hein Htet Aung..... 23
11.1.2 Felix Herrmann..... 23
11.2 Instances Where AI Was Incorrect or Unhelpful..... 23
11.2.1 Example 1..... 23
11.2.2 Example 2..... 23
11.2.3 Example 3..... 24
11.3 Reflection on AI in Group Work..... 24
12. Individual reflection..... 25
12.1 Individual Reflection - Hein Htet Aung..... 25
12.1.1 What's the worst case for your algorithm, and what kind of input triggers it?..... 25
12.1.2 What was the hardest bug or design decision you encountered, and how did you resolve it?..... 25
12.1.3 If you did this implementation again from scratch, what would you do differently?.. 25
12.1.4 What's one thing about your algorithm that you didn't fully understand before starting, and now do?..... 26
12.2 Individual Reflection - Felix Herrmann..... 27
12.2.1 What's the worst case for your algorithm, and what kind of input triggers it?..... 27
12.2.2 What was the hardest bug or design decision you encountered, and how did you resolve it?..... 27
12.2.3 If you did this implementation again from scratch, what would you do differently?.. 27
12.2.4 What's one thing about your algorithm that you didn't fully understand before starting, and now do?..... 27

1. The problem and the algorithms

1.1 The problem

Quicksort is a sorting algorithm designed to, as the name implies, quickly sort given inputs. It does so by recursively splitting the input into 2 inputs using a chosen “pivot” as the boundary. One of the split inputs contains all the numbers smaller than the pivot, and the other contains the inputs larger than the pivot. Because of the division around the pivot, these inputs when individually sorted will result in the original input being sorted. This divide and conquer strategy leads to an average runtime of $O(n\ log\ n)$. Quicksort unfortunately has a worst case runtime of $O(n^2)$ if bad pivots are selected, and thus pivot selection is a very important problem to solve when using the quicksort algorithm.

1.2 The algorithms

This project uses two versions of quicksort: last element deterministic pivot selection and uniformly randomized pivot selection. Both approaches employ the same divide-and-conquer algorithm, which iteratively separates a given array around a predetermined pivot until the entire array is sorted. The sole change is the selection technique of the pivot used at each recursive iteration.

Both solutions employ the Lomuto partitioning scheme to provide a meaningful comparison. Before the partitioning operation begins, the pivot is set to the subarray's final index between 'lowerLimit' and 'upperLimit'. A boundary index is started at 'lowerLimit', separating entries smaller than the pivot from those that have not yet been processed.

The method traverses the subarray and compares each element to the pivot using 'input[i] < pivot'. When this is true, the element is exchanged for the element at the boundary index, and the boundary index is increased. At the end of the loop, the pivot is swapped into the boundary position, resulting in its final sorted index.

This creates two partitions: items less than the pivot on the left and elements more than or equal to the pivot on the right. The method then sorts the two partitions recursively. The recursion ends when a partition is empty or contains only one element.

Both versions sort the input array in place and employ the identical comparison condition, splitting method, and recursive structure. As a result, the pivot selection approach is the primary variable explored in this research, with the goal of determining its impact on execution time, number of comparisons, number of swaps, recursion depth, and partition balance.

1.2.1 Deterministic pivot selection

The deterministic approach, developed by Felix Herrmann, uses a fixed pivot-selection rule, always selecting the last element of the current subarray as the pivot.

In this scenario, the pivot is already at the place predicted by the Lomuto partitioning technique, therefore no extra pivot selection or preliminary swap is necessary. The method will always use the same set of pivot choices and partitions on the same input array.

This approach is simple to implement and eliminates the computational expense of random number creation. However, its fixed selection rule makes it sensitive to the order of the input pieces. For example, if an array is already sorted in ascending order, continually selecting the final member leads in extremely lopsided divisions, with one partition comprising practically all of the remaining elements.

For some inputs, the deterministic last-element pivot selection takes $O(n^2)$. When the partitions are adequately balanced, it will take O(n log n).

Deterministic algorithms are more consistent in how they perform, but this means specific inputs can be created with far worse runtimes than average. Randomised algorithms by virtue of their randomness reduces the likelihood of consistently poor performance on specific inputs and in return have more uncertainty in their runtime.

1.2.2 Randomised pivot selection

Hein Htet Aung devised the random implementation, which uses a uniform random pivot selection instead of a predefined rule. At each recursive step, a random index is selected from the current subarray (including the lower and upper borders).

To choose the pivot index, the code employs the C++ Mersenne Twister pseudorandom number generator (std::mt 19937) with a specified seed and std::uniform_int_distribution. The selected element is swapped with the subarray's final element, followed by the usual Lomuto partitioning operation. Recursive calls use the same pseudorandom generator by reference, ensuring that the sequence of random selections made throughout the sort is unbroken.

Randomized pivot selection, unlike deterministic last-element selection, does not always match a specified input ordering to a predetermined series of unbalanced partitions. This decreases the likelihood of worst-case behavior with structured inputs such as sorted and reverse-sorted arrays.

The randomized approach has an estimated time complexity of O(n log n), but the worst-case complexity remains $O(n^2)$ if pivot choices continually produce imbalanced partitions. Random number generation and initial pivot swaps incur computational overhead, therefore there is a

trade-off between the expense of pivot selection and the lower likelihood of pathological partitioning.

2. Theoretical comparison

2.1 Asymptotic complexity

Both algorithms have identical asymptotic runtimes, the difference coming from specific inputs. For both algorithms, the worst case run time is $O(n^2)$, the average case run time is $O(n\ log\ n)$ and the best case run time is also $O(n\ log\ n)$.

2.2 Constant factor

Randomised has the extra constant factor of deciding the pivot through random number generation. Deterministic pivot selection does not have to do this. Additionally, after deciding the pivot, the randomised algorithm also has to swap it with the final element of the input to perform the Lomuto partitioning scheme. On the other hand, deterministic pivot selection does not have to perform any swaps because it always selects the final element of the input every time.

2.3 Memory characteristics

The memory characteristics of quicksort is decided by the partitioning algorithm that is used. To ensure consistency in testing we implemented the Lomuto algorithm for both deterministic and randomised pivots. The Lomuto algorithm is an in place algorithm and uses constant additional memory, equivalent to the size of one object in the array, which is used when swapping two elements during partitioning.

2.4 Input specific behaviour

The biggest difference in deterministic and random pivots comes from input specific behaviour, with last element deterministic pivots having an $O(n^2)$ worst case runtime on sorted (including reverse) inputs and random pivots having no predictable change in performance due to the input. The worst case runtime from deterministic pivot selection is because selecting the maximum or minimum value of the input splits it at the very beginning or end of the input, therefore no longer providing the benefits of divide and conquer.

3. Predictions

Below are our predictions of the faster algorithm for each input that we tested, as well as the average runtime for that input. We predict there will be no crossovers in mean runtime for any of the inputs.

3.1 Random

Winner: Tie
Deterministic runtime: $O(n \log n)$
Randomised runtime: $O(n \log n)$

3.2 Sorted

Winner: Randomised pivot
Deterministic runtime: $O(n^2)$
Randomised runtime: $O(n \log n)$

3.3 Reverse sorted

Winner: Randomised pivot
Deterministic runtime: $O(n^2)$
Randomised runtime: $O(n \log n)$

3.4 Nearly sorted

Winner: Randomised pivot
Deterministic runtime: Slightly faster than $O(n^2)$
Randomised runtime: $O(n \log n)$

3.5 Many duplicates

Winner: Tie
Deterministic runtime: Hein predicts $O(n^2)$, Felix predicts $O(n \log n)$
Randomised runtime: Hein predicts $O(n^2)$, Felix predicts $O(n \log n)$

3.6 All equal

Winner: Tie
Deterministic runtime: $O(n^2)$
Randomised runtime: $O(n^2)$

4. Experimental methodology

This experiment consists of two different quicksort implementations in which their pivot-selection strategy differs: Deterministic quicksort and Randomized quicksort.

- Deterministic quicksort: the last element is always selected as the pivot.
- Randomized quicksort: It selects a pivot uniformly at random from the current subarray, swaps its position with the last element of the array, and then works with the same partitioning scheme as the deterministic quicksort with minimum variabilities.

For a fair comparison, Lomuto partitioning scheme is implemented for both quicksort algorithms where the same comparison condition, less than pivot (< pivot), and the same recursive subproblem structure is implemented. Hence, the primary difference between the two implementations is the pivot-selection strategy.

4.1 Benchmark Inputs

The performance and stats of both algorithms were determined on six input types chosen to portray different quicksort behaviors:

1. Random - a shuffled array of the integers between 1 to n
2. Sorted - an array that is already sorted in ascending order from 1 to n
3. Reverse - an array sorted in descending order from n to 1
4. Nearly Sorted - a sorted array in which a small number (specifically n/4 elements) of elements were swapped with their adjacent neighbors
5. Many duplicates - an array containing multiple repeated values by being generated from a small value range
6. All equal - an array with the all elements being the same value

These input types were run at six array sizes:

- 100
- 1000
- 5000
- 10000
- 15000
- 20000

For input types that rely on randomness (specifically random, nearly sorted, and many duplicates), a set of distinct input seeds were added to ensure results were not dependent on a single generated array.

4.2 Repetitions, Seeds, and Warm-up

Fixed seeds were utilized to govern the benchmark, improving repeatability and reducing unintentional bias.

- Randomized Input Generation: Five input seeds (4020006181, 3572205801, 693963546, 1533451385, and 1727316755) were used.
- Randomized quicksort: Five pivot seeds were used: 4393318, 3257601471, 469755461, 804904386, and 3853312017.

The benchmark was conducted for each dataset configuration consisting of:

- Five untimed warm up rounds.
- Five repetitions were tested for timing performance.

The warm-up rounds were included after an initial time trend revealed that early repeats were frequently slower than later ones. The warm-up phase was designed to reduce the impact of starting difficulties such as cache warming, CPU frequency scaling, and other short-term system effects before assessing results.

Deterministic quicksort, while applicable, merely adjusts the input seed since pivot randomness is not used. In randomized quicksort, both the input seed and the pivot seed impact the run.

4.3 Timing Procedure

The execution time was solely assessed for the sorting process.

To isolate sorting time as much as possible,

- input generation was performed before timing,
- the original input array was copied into a working array before timing,
- correctness checking against std::sort() was performed after timing.

This prevents any distortion in the observed runtime caused by array generation, copying, and validation.

Timing was measured in milliseconds using std::chrono::steady_clock.

To minimize systematic order bias, the benchmark did not always execute the same method first. Instead, for each measured repeat, the benchmark generated a schedule that included:

- one deterministic run, and
- one randomized run for each pivot seed,

and then rearranged their execution sequences before running them. This helps to eliminate any advantage that may have been gained from executing the identical algorithm early or late.

4.4 Hardware and Compiler Configuration

All benchmarking experiments were conducted on a computers with the following specifications:

Component	Specification
Device	MacBook Pro 2024
Processor	Apple M3 Pro
Memory (RAM)	18 GB
Operating System	macOS 26.5.1
Programming Language	C++17
Compiler	Apple clang version 21.0.0 (clang-2100.1.1.101)
Build System	CMake version 4.4.3
Build Configuration	Release mode

To ensure consistency in the experimental context, both deterministic and randomized versions of quicksort were produced using the same compiler and optimization settings. The benchmark was compiled in Release mode to enable compiler optimizations and decrease debugging overhead.

We timed the execution time with std::chrono::steady_clock and translated the results to milliseconds. All trials were carried out on the same system to reduce hardware variations between the two implementations.

Both algorithms were executed under identical settings, however background processes, CPU frequency scaling, heat conditions, and operating system scheduling may have caused minor differences in measured execution durations.

4.5 Correctness Validation

We validated the accuracy of all benchmarked runs.

Each dataset was sorted using std::sort() to create a reference-sorted array. The results of each quicksort process were compared to the reference. The run was regarded as successful if the sorting results were precisely as predicted.

If a run fails checks for correctness:

- Its status was recorded as FAIL,

- It was excluded from statistical aggregation for timing and analysis metrics,
- But it was still counted in the dashboard's attempted/passed/failed totals.

Hence, incorrect runs do not influence the reported averages.

4.6 Additional Algorithmic Measurements

Apart from the execution time, alternate versions of both algorithms were also constructed to derived algorithm-level metrics:

- Number of element comparisons
- Number of swap operations
- Maximum recursion depth
- Initial partition imbalance

The collection of these metrics were designed to be done outside of the performance timing of the algorithm so that metric collection does not negatively impact and distort the runtime measurements.

4.6.1 Comparisons

Every time we encountered the following code,

input[i] < pivot ,

It is counted as a comparison.

4.6.2 Swaps

For any explicit call to std::swap (including self-swaps), it is counted as a swap. Therefore, the swap reported as a result also includes those operations executed by the implementation, but not just the swaps between distinct positions.

4.6.3 Maximum Recursion Depth

Recursion depth represents the deepest recursive call tracked during the sorts of both algorithms. The root call of the algorithm began at depth 1.

4.6.4 Initial Partition Imbalance

The first partition imbalance is the size difference between the two subarrays generated by quicksort's first partitioning operation. This measure only takes into account the initial division of the original input array, rather than the partition imbalance that occurs throughout the recursive sorting process.

The imbalance is calculated using:

$$Initial\ Imbalance\ = \frac{|L-R|}{L+R}$$

Where:

- L is the size of the left partition at depth = 1,
- R is the size of the right partition at depth = 1.

The value of the imbalance can be between a range of 0 to 1 inclusive, where:

- 0 indicates a perfectly balanced partition,
- 1 indicates a completely unbalanced partition.

The initial imbalance for randomized quicksort might vary depending on the pivot seeds. As a result, for each input type, method, and array size group, the dashboard shows the average initial partition imbalance over successful analysis runs.

Each iteration of the analysis simply saves the imbalance caused by the first split of the original input array. The dashboard then averages the initial imbalance readings for each input type, method, and array size group over many successful analysis runs. There are no imbalance observations from further recursive partitioning.

4.7 Output and Aggregation

Two levels of outputs were generated by this benchmark algorithm.

1. Detailed CSV files - one csv file per each input type, consisting of the results of each individual run.
2. Dashboard CSV - an aggregated summary grouped by:
    a. Input type,
    b. Algorithm, and
    c. Array Size.

For performance timing, the dashboard displays:

- Sample count,
- Mean time,
- Median time,
- Standard deviation,
- Minimum time, and
- Maximum time.

For algorithmic analysis, the dashboard portrays:

- Number of successful analysis samples,
- Mean comparisons,
- Mean swaps,
- Mean maximum recursive depth, and
- Mean initial partition imbalance.

The dashboard also shows other data like:

- Attempted runs,
- Passed runs, and
- Failed runs.

4.8 Experimental Scope and Limitations

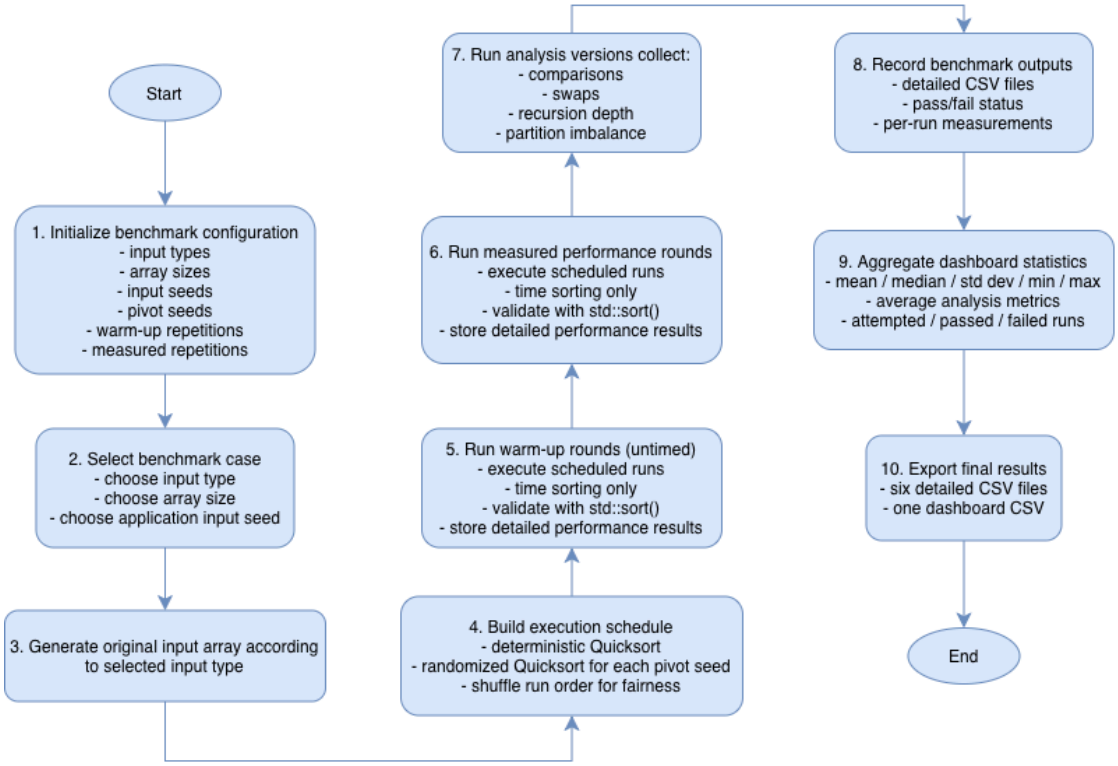
This experiment examines two entire quicksort implementations, not simply the asymptotic behavior. Thus, the measured running time incorporates the practical expenses of implementation, such as random pivot selection and random number creation in the randomized variant.

Both techniques employ a two-way partitioning strategy based on the condition “< pivot”. Thus, even with the randomized variant, arrays with numerous duplicate or equivalent entries may result in uneven partitions. This is not an implementation issue, but rather an algorithmic restriction of the shared partition architecture.

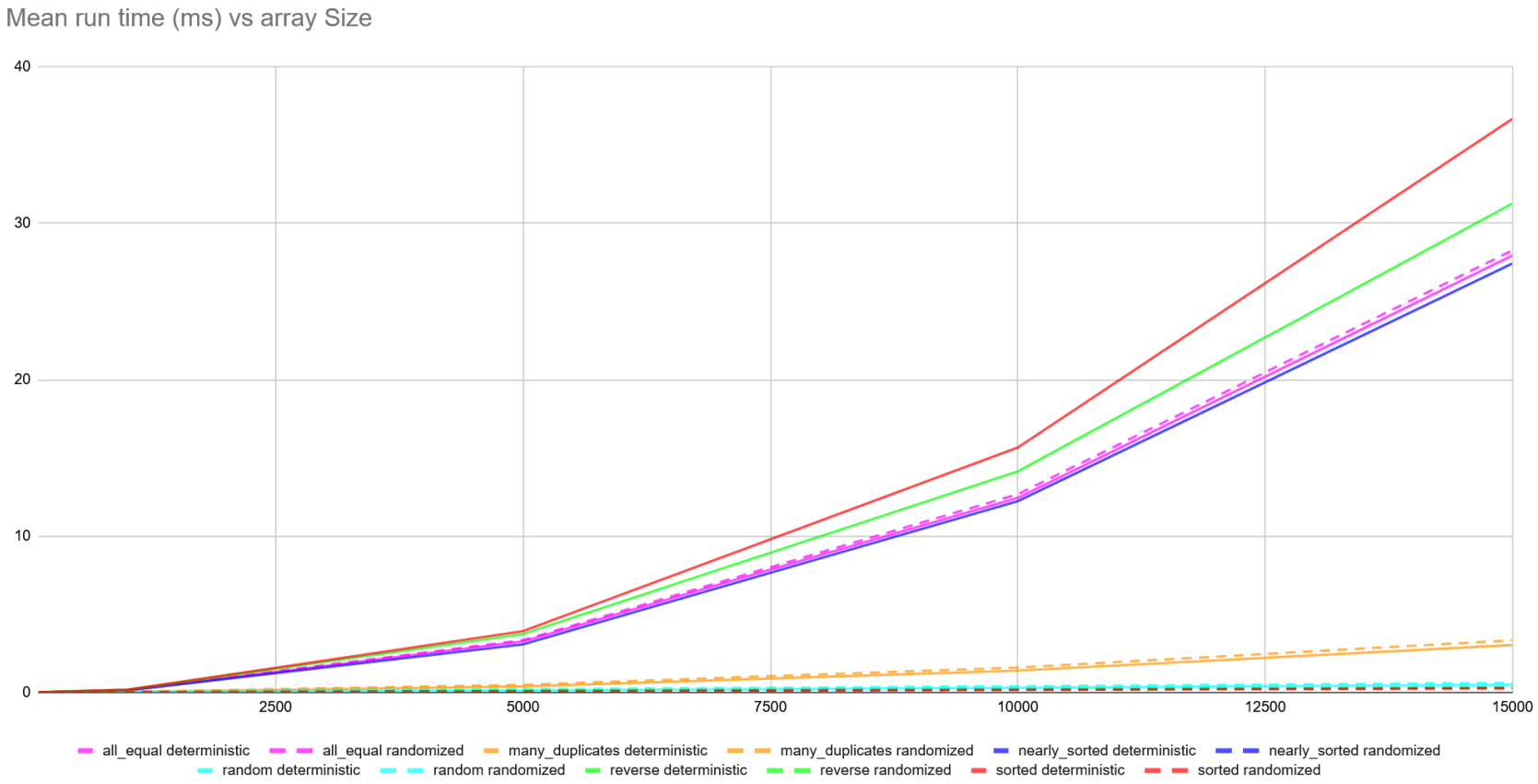
Finally, warm-up runs and repeats improve dependability. However, runtime measurements on extremely tiny inputs are vulnerable to system noise and should be interpreted with caution.

4.9 Flow Diagram of the Experiment algorithm

Figure 1. Benchmark workflow for deterministic and randomized quicksort Comparison



5. Results and analysis



Solid lines are deterministic and dotted lines are randomised

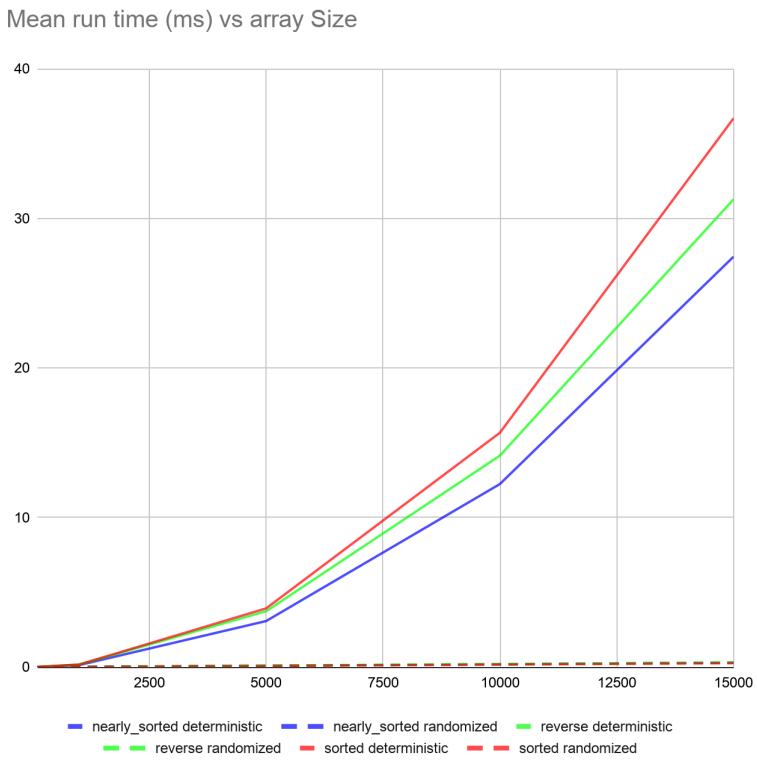
The graph of mean run time vs array size for all tests is a great overview of how the algorithms performed in different benchmarks, although it hides differences between the lower runtime tests. The main takeaways are:

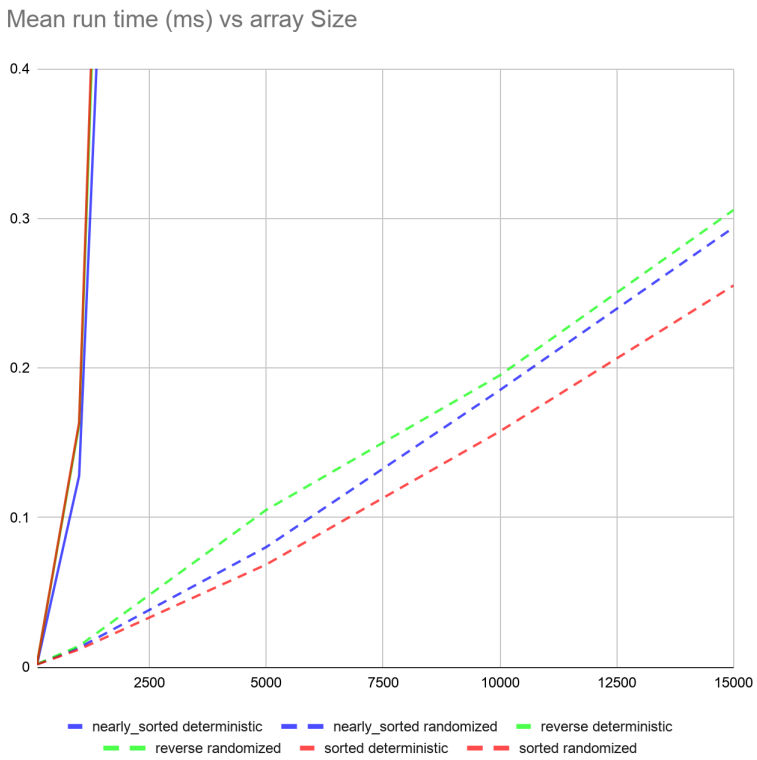
5.1 Deterministic pivots on sorted inputs

The deterministic algorithm when used on an already sorted list (or even mostly sorted in the case of the nearly sorted benchmark) runs significantly slower than on random inputs, confirming our theoretical understanding of worst case runtimes and revealing that the algorithm is still vulnerable to lists that aren't perfectly sorted.

5.1.1 Similarity of deterministic and randomised algorithm within the same test

5.1.1.1 Significantly different: sorted, reverse sorted, nearly sorted

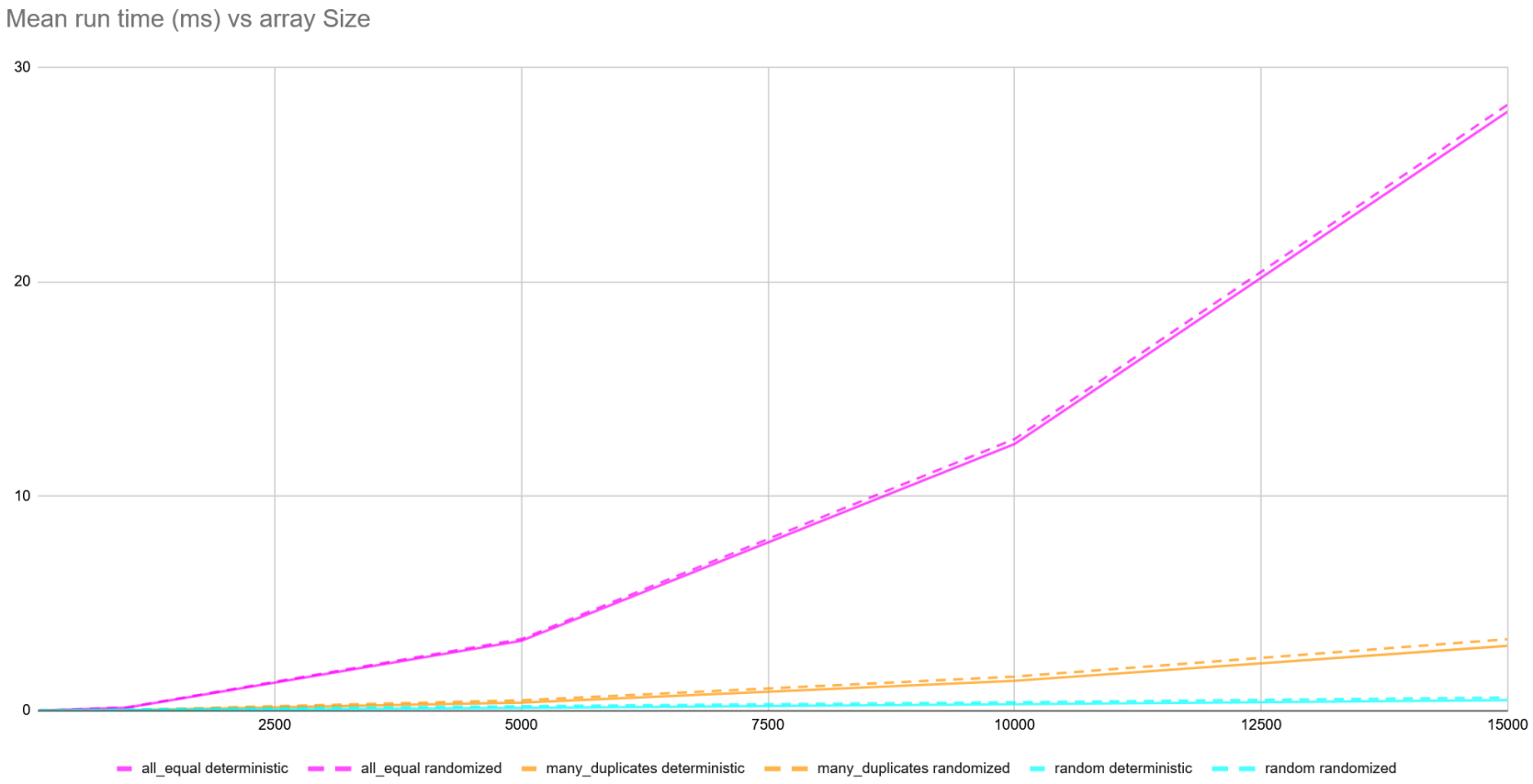




Graphs of only tests with significantly different mean run times between deterministic and randomised pivot selection
Left graph is scaled to fit all data, right graph is scaled to show smaller values

These tests show the failure of deterministic pivot selection fails and success of randomised pivot selection and demonstrate the input agnostic nature of randomised pivot selections' performance. This makes sense as it is very unlikely for a randomised algorithm to make the optimally bad choices that deterministic selection does leading to unnecessary recursion depth without effectively splitting the input.

5.1.1.2 Mostly similar: all equal, many duplicates, random



Graph of only tests with mostly similar mean run times between deterministic and randomised pivot selection

These tests show when both algorithms fail or succeed together. Random inputs are the optimal input for both algorithms and both perform well. It might be expected for many duplicates to be slower for deterministic pivot selection but it remains faster and both algorithms are similar. This suggests that the inputs were still random enough for both to succeed but did not allow for deterministic pivot selection to choose consistently bad pivots. All equal inputs is effectively an already sorted array, when doing a comparison sort and so deterministic sort is slow, but randomised pivot selection is no longer random and chooses the same pivot value as

deterministic pivot selection. Randomised pivot selection performed slightly worse in all of these tests because its randomness did not allow it to make better decisions than deterministic pivot selection, and the generation of pseudorandom numbers is performance intensive.

5.1.2 Two tight runtime groupings

The runtimes of all algorithms roughly fell into the expected $O(nlogn)$ and $O(n^2)$ groupings, showing that in the tests we gave there was very little middle ground and quicksort either performed very well or terribly. Finding scenarios that blur this boundary is worth investigating further in the future.

5.2 Algorithmic Measurements: Explaining the Observed Performance

The extra algorithmic measurements give greater insight into the observed execution-time disparities between deterministic and randomized Quicksort.

For an array of 20,000 elements, deterministic Quicksort performed 199,990,000 comparisons on sorted data, whereas randomized Quicksort executed around 334,785 comparisons. The greatest recursion depth was 20,000, with a mean of 32.8 for the randomized implementation. A similar pattern was noted with almost sorted inputs, when deterministic Quicksort completed roughly 155.94 million comparisons and had an average maximum recursion depth of 15,614. Randomized Quicksort completed roughly 337,265 comparisons, and had an average maximum depth of 33.9. These measurements explain the huge runtime discrepancies that we find in the performance graphs. Deterministic last-element pivot selection creates highly imbalanced recursive partitions on certain structured inputs, resulting in many more comparisons and deeper recursion.

However, the all-equal input illustrates a shortcoming shared by both systems. Both conducted precisely 199,990,000 comparisons and attained a maximum recursion depth of 20,000. Under the shared Lomuto partitioning system, random pivot selection cannot enhance the partition balance, as all elements have the same value.

The swap metrics further indicate that runtime is not entirely defined by comparison tallies. Deterministic Quicksort on inputs sorted in ascending order made 200,009,999 swaps (including self-swaps), but inputs with all equal values needed just 19,999 swaps, although the number of comparisons was the same. Average execution times were roughly 95.34 and 49.63 ms, respectively.

Finally, initial partition imbalance aids the understanding of the structured-input outcomes. For 20,000 items the mean initial imbalance for deterministic Quicksort was found to be 1.0 on nearly sorted inputs, but randomized Quicksort had a mean imbalance of roughly 0.412. This measurement just assesses the first partition, hence it should be evaluated in conjunction with comparisons and recursion depth rather than as a comprehensive indicator of partitioning behavior.

The supplementary measurements indicate that the variations in runtime are significantly associated with the partition balance, recursion depth, and the quantity of operations executed in each implementation.

5.3 Prediction vs reality

We both agreed that for the many inputs input, the algorithms would have the same average runtime but disagreed on what that runtime would be, with Hein predicting $O(n^2)$ and Felix predicting $O(nlogn)$. The test ended up running slightly slower than $O(nlogn)$ for both algorithms but this does not provide a clear answer to our hypotheses, as the density of the duplicates could have a massive effect on the runtime. Additionally our test was flawed, with the density of the duplicates being variable for different input sizes. The all equal test is actually also a many duplicates test with all inputs being the same, and both algorithms ran in $O(n^2)$.

All of the tests we predicted a tie for were very close but the randomised algorithm was consistently slightly slower than the deterministic algorithm which we didn't include in our predictions.

6. What surprised us

Our research yielded two surprising results concerning quicksort's running time. These findings prompted us to investigate our benchmarking approach and the correlation between input structure and algorithm performance.

6.1 Decreasing Execution Time Across Repetitions

One interesting conclusion was that execution time decreased with consecutive repeats of the same benchmark setting. Initially, we anticipated the execution time to be the same when the same algorithm was run on the same input data and under the same conditions. However, earlier repeats sometimes required longer execution durations than later repetitions.

One probable cause is the influence of hardware and execution environment effects, such as CPU cache warming and dynamic processor frequency modifications. When an instruction is performed for the first time, the execution time may be longer. The instructions and data may not yet be in the processor's cache. Later repeats might benefit from previously available instructions and data.

This pushed us to reconsider our benchmarking process and include untimed warm-up rounds prior to gathering performance measures. Warm-ups cannot eliminate the effects of caching or other environmental fluctuations, but they can mitigate the influence of initial execution circumstances on recorded outcomes.

The finding revealed that the measured execution time is determined not just by an algorithm's theoretical complexity, but also by experimental circumstances.

6.2 Near-Quadratic Performance on Nearly Sorted Inputs

It was also shown that deterministic quicksort may accomplish almost quadratic time complexity, $O(n^2)$, even if the input array is not fully sorted.

Our initial idea was that interrupting the sorting order will minimize the severity of the imbalanced partitions caused by deterministic last-element pivot selection. Despite the largely sorted inputs, the execution durations were consistent with the algorithm's worst-case behavior.

One potential explanation is that the input creation technique swaps neighboring elements on an initially sorted array. These exchanges cause local disarray, but most of the ordering persists. So using the final element as the pivot might still result in severely uneven divisions down the recursion line.

As a result, adding a modest bit of disorder to a sorted array may not be enough to prevent poor performance in deterministic pivot selection.

The discovery also raises a challenge for additional investigation: how much disturbance of the original ordering is required before a deterministic quicksort approaches its predicted average-case behavior?

7. What we'd do next

The experiment provided some insight into the performance differences between quicksort's deterministic and randomized pivot selection versions, but many issues remain for further exploration.

7.1 Alternative deterministic pivot-selection strategies

We presently have a deterministic implementation that uses the last element as the pivot. An easy expansion would be to include more deterministic pivot selection procedures, such as first-element, middle-element, and median-of-three. It would be interesting to compare these techniques to uniformly randomized pivot selection to see if the performance gains reported with randomized quicksort are mostly attributable to randomization or the avoidance of consistently bad pivot choices.

7.2 Alternative partitioning schemes

Currently, both implementations employ the Lomuto partition technique to ensure consistency. Future research can focus on Hoare partitioning and three-way partitioning, particularly for inputs with a high number of duplicate or identical items. A three-way partition is particularly intriguing since it divides the elements into three categories: less than the pivot, equal to the pivot, and more than the pivot. This might be a remedy to the poor performance of both implementations on inputs with several duplicates.

7.3 Varying the degree of sortedness

Our input generator now performs (n/4) adjacent swaps for a nearly sorted array. Future research might include various swap configurations like (n/16), (n/8), (n/4), and (n/2). This would allow us to investigate the impact of progressively modifying the original order on execution time, comparisons, recursion depth, and initial partition imbalance. It may also provide a threshold for when deterministic quicksort competes with randomized quicksort.

7.4 Investigating duplicate density

Our current duplicate-heavy input generator selects values from a defined range of 0 to 9 regardless of array size, resulting in a high number of duplicates, especially in tests with large array sizes. Future tests might alter the range of values to see how increasing or reducing duplicate density affects partition balance and performance. This may provide further information on the limits of the shared two-way partitioning strategy. Variably changing the range of values based on array size will help to show how the algorithm scales without being affected by the scaling of duplicate density that our current benchmark has.

7.5 Examining the relationship between initial partition and overall performance

Our approach preserves the initial partition's imbalance, comparisons, swaps, and maximum recursion depth. An addition may investigate if the entire execution time is highly connected to the imbalance of the initial partition or if future recursive partitions have a greater impact. This may help us determine how indicative of overall quicksort performance the initial partition imbalance is.

7.6 Expanding performance measurements

Finally, future research may incorporate greater input sizes, more repetitions, and testing with other hardware and compiler setups. This would aid in determining if the observed performance differences are constant across multiple execution contexts and give better evidence of variability in randomized QuickSort.

Overall, these modifications would allow us to investigate the interplay between pivot selection, input characteristics, and partitioning methods in their influence on quicksort performance, rather than just comparing two pivot-selection strategies.

8. Individual contributions

Member	Contributions
Hein Htet Aung 25026511	- Implementation of Randomised pivot selection in Quicksort by adapting to Felix's Deterministic pivot selection with minimal variations between the implementations - Planning and implementation of Benchmarking algorithm - Experimental Methodology, What surprised us, what we'd do next and AI Disclosure sections in the report - Github repository and report structure setup - Adapting the benchmarking algorithm to new findings in the results - Individual slides for the presentation - Video editing, YouTube Upload and submission
Felix Herrmann 25483227	- Implementation of Deterministic pivot selection in Quicksort - Boilerplate used for both algorithms including test inputs read from files, print functions, etc. - Assisted in the Benchmarking algorithm planning - Modifications to benchmarking: number of samples, more random seeds, random seed generation. - The problem definition, theoretical comparison, predictions, and results and analysis sections in the report. - Analysis of csv results, production of graphs, contributing to the groups understanding of how both algorithms compare in different scenarios. - Individual slides for the video presentation.

9. GitHub Repository Link

https://github.com/Hein-Htet-Aung08/AdvAlgoA2

10. Video Link

https://youtu.be/aSnGIHinGY0

11. AI use

11.1 Individual AI Use Disclosure

11.1.1 Hein Htet Aung

I used ChatGPT throughout the project as a development assistant, primarily for providing suitable pseudocode and refining my randomised quicksort algorithm, brainstorming the unified benchmarking framework, and reviewing the experimental methodology. AI was also used to assist with the refinement of certain sections in the written report as well as to check whether all required information is written.

During implementation, I used ChatGPT to extend and brainstorm my initial ideas, discuss possible approaches, identify potential flaws, and explain unfamiliar or misunderstood concepts. Every suggestion provided is evaluated carefully, adapted, rejected or reviewed, compiled and executed the benchmark, and analyze and access the generated results to verify correctness.

In this project, all AI explanations and suggestions are properly thought through and accounted without any of them treated as automatically correct.

11.1.2 Felix Herrmann

Felix did not use AI tools for his individual contributions to this project. His deterministic quicksort implementation and associated work were completed without direct AI assistance.

11.2 Instances Where AI Was Incorrect or Unhelpful

11.2.1 Example 1

While adjusting the benchmarking framework to keep unsuccessful configurations on the dashboard, the pseudocode had a redundant declaration of the attempted variable. The variable was previously introduced by a structured binding in a range-based loop, but it was then redeclared in the same scope. This resulted in a compilation fault.

The compiler's diagnostic messages indicated the problem, and the remedy was to remove the duplicate declaration. This proved that AI-assisted code modifications still require compilation and thorough evaluation, especially when modifying existing code.

11.2.2 Example 2

When constructing the dashboard reporting function, the CSV header had a field for analysis_samples, but the value was not initially put in each row. As a result, the data shown

below would have been recorded under the incorrect column headings, potentially leading to misleading statistical findings.

The discrepancy was discovered following a source code examination. Fixed the output logic so that the analysis sample count appears before the other analysis metrics. This underlined the need of verifying both the algorithm's accuracy and the integrity of the experimental data produced.

11.2.3 Example 3

When we changed the analytical metric from mean partition imbalance over recursive partitions to beginning partition imbalance, AI-assisted evaluation found anomalies in the detailed CSV and dashboard column names. The detailed findings were labeled as mean initial partition imbalance, while the dashboard utilized the term initial partition imbalance, despite the fact that the dashboard averaged information from numerous analysis runs.

We also changed the language to distinguish between the imbalance of the first partition within a single run and the average of those initial partitions for various input and pivot seeds. This demonstrated how contradictions might be produced or overlooked when an experimental measurement is redefined in the midst of AI-assisted explanations and code modifications.

11.3 Reflection on AI in Group Work

AI was particularly valuable for evaluating implementation possibilities, troubleshooting integration challenges, and developing benchmarking infrastructure. This was another source of technical input as we went over the code and explained the ramifications of design decisions like isolating instrumented analysis from performance timing and included warm-up repeats.

However, AI has increased the risk of embracing solutions that appear logical but have not been adequately verified. Some of the recommendations were missing or incompatible with the present implementation, necessitating extensive debugging, compilation, and scrutiny of the benchmark results. This emphasized the significance of approaching AI as an assistance rather than an authority.

This disparity in AI use among group members underscored the necessity for unambiguous individual accountability. Felix created the deterministic implementation on his own, but I employed assisted artificial intelligence to create the randomized implementation and benchmarking framework. However, understanding both algorithms was a shared obligation for ensuring a fair comparison.

Overall, AI proved most useful when used interactively to assess ideas and solve issues, rather than merely accepting generated code or explanations. The experiment demonstrated that while artificial intelligence may assist to accelerate development and improve comprehension, its effective use still requires human judgment, technological verification, and clear attribution.

12. Individual reflection

12.1 Individual Reflection - Hein Htet Aung

12.1.1 What's the worst case for your algorithm, and what kind of input triggers it?

The worst case time complexity for my randomized Quicksort implementation is $O(n^2)$. This worst case occurs when the pivot selections result in very unbalanced partitions, such that one partition is empty and the other partition contains almost all the remaining elements. The random selection of the pivot reduces the likelihood of this happening for sorted or reverse-sorted inputs, but it does not eliminate it. An array of identical elements is a useful worst-case input. In our solution, we employ Lomuto partitioning with stringent comparison input[i] < pivot on an input where all elements are the same, which prevents entries equal to the pivot from moving to the left partition. As a result, practically all remaining pieces are on one side of each partition, and the performance is quadratic regardless of the random pivot selection method.

12.1.2 What was the hardest bug or design decision you encountered, and how did you resolve it?

The hardest aspect of this project was designing the entire benchmarking system. The challenge was not only to construct the benchmark in C++, but also to design the experiment so that a fair and relevant comparison could be made. I had to consider how to build input data sets, how many seeds and repeats were suitable, how to reduce bias from order of execution, and how to separate performance timing from correctness testing and algorithmic instrumentation. One particularly challenging decision was to include warm-up rounds after seeing that earlier repeats sometimes took longer than later ones. This taught me that even a properly built benchmark might produce deceptive results if the experimental approach is not adequately structured. I addressed these concerns by progressively upgrading the benchmark, which included fixed seeds for repeatability, five untimed warm-up rounds, five timed repeats, a randomized execution order, and separate instrumented analysis runs. I also validated the sorting against std::sort() and examined the resulting results. The most difficult element was developing an experimental procedure that I could defend, rather than simply writing code that executed both algorithms.

12.1.3 If you did this implementation again from scratch, what would you do differently?

In retrospect, if I were to start this project over, I would spend more time preparing the benchmarking framework before completely implementing it. I wouldn't try to construct and polish a lot of things at once. I'd create a smaller experimental process, test it, and then

gradually add features such as different input seeds, repetition, warm-up rounds, algorithmic measures, and so on. I would also describe the experimental metrics and CSV output format earlier, as changes in measurement definitions throughout construction necessitated modifications to many aspects of the implementation. This would make the framework easier to debug, reduce the number of rewrites required, and allow me to spend more time evaluating the findings.

12.1.4 What's one thing about your algorithm that you didn't fully understand before starting, and now do?

Until I started this project, I had no idea how Lomuto's partitioning system worked or why its comparison condition and border index were so critical. Implementing it taught me how to maintain the partitioning invariant, shift items smaller than the pivot to one side, and bring the pivot to its final sorted location. In addition, I learnt more about how input factors influence the effectiveness of sorting algorithms in practice. I used to consider algorithm efficiency primarily in terms of theoretical time complexity. However, this experiment demonstrated that two implementations with identical asymptotic complexity might behave quite differently depending on the sequence of the inputs, the amount of duplicate values, and the pivot selection approach. In particular, I learnt why deterministic last-element pivot selection performs poorly on sorted inputs, and why randomization cannot save the day with two-way partitioning when all of the elements are identical. During this study, I was able to apply theoretical complexity analysis to actual algorithm behavior and experimental observations.

12.2 Individual Reflection - Felix Herrmann

12.2.1 What's the worst case for your algorithm, and what kind of input triggers it?

The worst case for deterministic last element pivot selection is a runtime on n^2 and is triggered by any kind of sorted or semisorted input, including inputs that cannot be unsorted, like in the all equal elements benchmark. These inputs undermine the divide and conquer nature of this algorithm. Sorted inputs are divided in a way that creates 1 input with an input size that is only 1 less. Semisorted inputs divide the input with varying levels of unevenness depending on how sorted they are, with more uneven partitions performing closer to n^2.

12.2.2 What was the hardest bug or design decision you encountered, and how did you resolve it?

The hardest bug I encountered was a single missing ‘&’ in the quicksort parameters to ensure I was passing a reference to the original input. The algorithm would successfully partition at the first level of depth because I was storing the returned input when I first called the quicksort but I did not store the return of recursive calls anywhere within the quicksort. I spent a very long time debugging by analysing debug outputs of the current state of the input and where in the stack the algorithm was. I eventually discovered that the recursive partitions were correctly sorting the input but this sorting was immediately reverted after returning. Seeing this helped me figure out that there must be something wrong when returning, and this was indeed the case. Adding a ‘&’ in front of the input parameter solved the problem.

12.2.3 If you did this implementation again from scratch, what would you do differently?

I wouldn’t have the quicksort return anything in the first place when the algorithm is meant to be in-place, I would have noticed my mistake much earlier but storing the return of the quicksort allowed the successful first partition to cover up mistakes with the recursion.I would also make a helper function for partitioning, it makes the code much easier to read.

12.2.4 What's one thing about your algorithm that you didn't fully understand before starting, and now do?

I did not understand the Lomuto algorithm at all and only had a theoretical understanding of partitioning and the basic idea of the naive partition that creates 2 new arrays to store values in. I now fully understand how the array is traversed, input elements are swapped with the boundary, and how the pivot is then swapped at the end.